

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	B V Chaitanya Reddy	Reg. No.:	192425376
Section / Batch:	2024	Date:	

Code of Conduct

I B V Chaitanya(192425376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: chaitanya

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.

7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	3	3	3	4	4	17	81
2	3	3	3	3	3	15	
3	3	3	3	4	4	17	
4	3	3	3	3	3	15	
5	3	3	3	4	4	17	

Faculty In-charge	Course Coordinator	Program Director

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 – Biomedical Morphological Analysis

This program analyzes treatment, treatable, retreatment, treated, and untreated and identifies prefixes, roots, suffixes, and morphological types.

Code:

Q1: Biomedical Morphological Analysis

```
import re
```

```
words = [
    "treatment",
    "treatable",
    "retreatment",
    "treated",
    "untreated"
]
```

Morphological analysis rules

```
def analyze_word(word):
```

```
    original = word
```

```
    prefix = ""
```

```
    root = word
```

```
    suffix = ""
```

```
    features = []
```

```
    # Prefix analysis
```

```
    if root.startswith("re"):
```

```
        prefix = "re-"
```

```
        root = root[2:]
```

```
        features.append("derivational prefix: re-")
```

```
    elif root.startswith("un"):
```

```
        prefix = "un-"
```

```
        root = root[2:]
```

```
        features.append("derivational prefix: un-")
```

```
    # Suffix analysis
```

```
    if root.endswith("ment"):
```

```
        root = root[:-4]
```

```
        suffix = "-ment"
```

```
        features.append("derivational suffix: -ment")
```

```
    elif root.endswith("able"):
```

```

    root = root[:-4]
    suffix = "-able"
    features.append("derivational suffix: -able")

elif root.endswith("ed"):
    root = root[:-2]
    suffix = "-ed"
    features.append("inflectional suffix: -ed")

# Handle possible spelling adjustment
if root == "treat":
    base = "treat"
else:
    base = root

return {
    "Word": original,
    "Prefix": prefix if prefix else "None",
    "Root": base,
    "Suffix": suffix if suffix else "None",
    "Features": ", ".join(features) if features else "None"
}

```

```

results = [analyze_word(word) for word in words]

```

```

for result in results:
    print("Word:", result["Word"])
    print("Prefix:", result["Prefix"])
    print("Root:", result["Root"])
    print("Suffix:", result["Suffix"])
    print("Analysis:", result["Features"])
    print("-" * 50)

```

Output:

```

... Word: treatment
    Prefix: None
    Root: treat
    Suffix: -ment
    Analysis: derivational suffix: -ment
    -----
    Word: treatable
    Prefix: None
    Root: treat
    Suffix: -able
    Analysis: derivational suffix: -able
    -----
    Word: retreatment
    Prefix: re-
    Root: treat
    Suffix: -ment
    Analysis: derivational prefix: re-, derivational suffix: -ment

```

Q2 – Finite-State Morphological Parser

This program demonstrates the incorrect parser, corrected parser, multiple prefixes/suffixes, and accuracy calculation.

Code:

```

words = [
    "replayed",
    "unhappier",
    "disconnected",
    "players",
    "restarting",
    "unreadable"
]

# -----
# Incorrect parser
# -----

def incorrect_parser(word):

    prefix = ""
    root = word
    suffix = ""

    # Recognizes only one prefix
    if root.startswith("re"):
        prefix = "re-"
        root = root[2:]
    elif root.startswith("un"):
        prefix = "un-"
        root = root[2:]
    elif root.startswith("dis"):
        prefix = "dis-"

```

```
root = root[3:]
```

```
# Recognizes only one suffix
```

```
if root.endswith("ed"):
```

```
    suffix = "-ed"
```

```
    root = root[:-2]
```

```
elif root.endswith("s"):
```

```
    suffix = "-s"
```

```
    root = root[:-1]
```

```
elif root.endswith("ing"):
```

```
    suffix = "-ing"
```

```
    root = root[:-3]
```

```
elif root.endswith("able"):
```

```
    suffix = "-able"
```

```
    root = root[:-4]
```

```
return prefix, root, suffix
```

```
# -----
```

```
# Corrected FST parser
```

```
# -----
```

```
def corrected_parser(word):
```

```
    prefixes = []
```

```
    suffixes = []
```

```
    root = word
```

```
# Multiple prefix transitions
```

```
prefix_rules = ["un", "re", "dis"]
```

```
changed = True
```

```
while changed:
```

```
    changed = False
```

```
    for p in prefix_rules:
```

```
        if root.startswith(p):
```

```
            prefixes.append(p + "-")
```

```
            root = root[len(p):]
```

```
            changed = True
```

```
            break
```

```
# Suffix transitions
```



```
suffix_rules = ["ing", "ed", "er", "able", "s"]
```

```
changed = True
```

```
while changed:  
    changed = False
```

```
    for s in suffix_rules:  
        if root.endswith(s) and len(root) > len(s):  
            suffixes.insert(0, "-" + s)  
            root = root[:-len(s)]  
            changed = True  
            break
```

```
return prefixes, root, suffixes
```

```
print("INCORRECT PARSER OUTPUT")  
print("=" * 60)
```

```
for word in words:  
    print(word, "->", incorrect_parser(word))
```

```
print("\nCORRECTED FST PARSER OUTPUT")  
print("=" * 60)
```

```
for word in words:  
    prefix, root, suffix = corrected_parser(word)
```

```
    print("Word:", word)  
    print("Prefixes:", prefix if prefix else "None")  
    print("Root:", root)  
    print("Suffixes:", suffix if suffix else "None")  
    print("-" * 60)
```

```
# -----  
# Expected morphological analyses  
# -----
```

```
expected = {  
    "replayed": (["re-"], "play", ["-ed"]),  
    "unhappier": (["un-"], "happi", ["-er"]),  
    "disconnected": (["dis-"], "connect", ["-ed"]),  
    "players": ([], "player", ["-s"]),
```

```

    "restarting": (["re-"], "start", ["-ing"]),
    "unreadable": (["un-"], "read", ["-able"])
}

```

```

# -----
# Accuracy calculation
# -----

```

```
correct = 0
```

```
for word in words:
```

```

    predicted = corrected_parser(word)
    actual = expected[word]

```

```

    if predicted == actual:
        correct += 1

```

```
accuracy = (correct / len(words)) * 100
```

```

print("\nPARSING ACCURACY")
print("=" * 60)
print("Correct analyses:", correct)
print("Total words:", len(words))
print("Accuracy:", accuracy, "%")

```

Output:

```

Word: restarting
Prefixes: ['re-']
Root: start
Suffixes: ['-ing']
-----
Word: unreadable
Prefixes: ['un-', 're-']
Root: ad
Suffixes: ['-able']
-----

PARSING ACCURACY
=====
Correct analyses: 4
Total words: 6
Accuracy: 66.66666666666666 %

```

Q3 – News Document Tokenization and Porter Stemming

This is the most important one for the programming question because the original program incorrectly sends an entire sentence to `ps.stem()`.

code:

```

import nltk
import pandas as pd
import re

from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

nltk.download("punkt")
nltk.download("punkt_tab")

ps = PorterStemmer()

# -----
# Sample news dataset
# -----

data = pd.DataFrame({
    "Text": [
        "The government announced new economic policies today.",
        "Scientists developed a new treatment for the disease.",
        "The football team won the championship yesterday.",
        "Technology companies are developing artificial intelligence systems.",
        "The central bank increased interest rates."
    ]
})

# -----
# Incorrect processing
# -----

data["Original_Output"] = data["Text"].apply(ps.stem)

print("INCORRECT OUTPUT")
print("=" * 70)

for text, output in zip(data["Text"], data["Original_Output"]):
    print("Original:", text)
    print("Incorrect:", output)
    print("-" * 70)

# -----
# Correct tokenization + stemming
# -----

```

```

def preprocess(text):

    # Convert to lowercase
    text = text.lower()

    # Tokenization
    tokens = word_tokenize(text)

    # Keep alphabetic tokens only
    tokens = [
        word for word in tokens
        if word.isalpha()
    ]

    # Apply Porter stemming to each token
    stems = [
        ps.stem(word)
        for word in tokens
    ]

    return tokens, stems

data["Tokens"] = data["Text"].apply(
    lambda x: preprocess(x)[0]
)

data["Stemmed_Tokens"] = data["Text"].apply(
    lambda x: preprocess(x)[1]
)

print("\nCORRECTED OUTPUT")
print("=" * 70)

for i, row in data.iterrows():

    print("Original Text:")
    print(row["Text"])

    print("\nTokens:")
    print(row["Tokens"])

    print("\nStemmed Tokens:")
    print(row["Stemmed_Tokens"])

```

```
print("-" * 70)
```

Output:

```
INCORRECT OUTPUT
=====
... Original: The government announced new economic policies today.
    Incorrect: the government announced new economic policies today.
    -----
    Original: Scientists developed a new treatment for the disease.
    Incorrect: scientists developed a new treatment for the disease.
    -----
    Original: The football team won the championship yesterday.
    Incorrect: the football team won the championship yesterday.
    -----
    Original: Technology companies are developing artificial intelligence systems.
    Incorrect: technology companies are developing artificial intelligence systems.
    -----
    Original: The central bank increased interest rates.
    Incorrect: the central bank increased interest rates.
    -----

CORRECTED OUTPUT
=====
Original Text:
The government announced new economic policies today.

Tokens:
['the', 'government', 'announced', 'new', 'economic', 'policies', 'today']

Stemmed Tokens:
['the', 'govern', 'announc', 'new', 'econom', 'polici', 'today']
-----
Original Text:
Scientists developed a new treatment for the disease.
```

Q4: Finite-State Morphological Parser for Plural Nouns

Code:

```
words = [
```

```
    "cars",
    "boxes",
    "cities",
    "children",
    "books",
    "dogs",
    "classes",
    "babies",
    "men",
    "women"
```

```
]
```

```
# -----
```

```
# Original / Incorrect Parser
```

```
# -----
```

```

def original_parser(word):

    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

print("ORIGINAL PARSER")
print("=" * 60)

for word in words:
    print(word, "->", original_parser(word))

# -----
# Corrected parser
# -----

irregular_plurals = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "geese": "goose",
    "feet": "foot",
    "teeth": "tooth"
}

def corrected_parser(word):

    # Irregular plural
    if word in irregular_plurals:
        return irregular_plurals[word], "Plural Noun", "Irregular plural"

    # -ies rule
    if word.endswith("ies"):
        root = word[:-3] + "y"
        return root, "Plural Noun", "-ies -> -y"

    # -es rule
    if word.endswith("es"):
        root = word[:-2]
        return root, "Plural Noun", "-es"

```

```
# Regular -s rule
if word.endswith("s"):
    root = word[:-1]
    return root, "Plural Noun", "-s"
```

```
# Singular
return word, "Singular Noun", "No plural suffix"
```

```
print("\nCORRECTED PARSER")
print("=" * 60)
```

```
for word in words:
```

```
    root, category, rule = corrected_parser(word)
```

```
    print("Word:", word)
    print("Root:", root)
    print("Category:", category)
    print("Rule:", rule)
    print("-" * 60)
```

```
# -----
# Accuracy
# -----
```

```
expected = {
    "cars": "car",
    "boxes": "box",
    "cities": "city",
    "children": "child",
    "books": "book",
    "dogs": "dog",
    "classes": "class",
    "babies": "baby",
    "men": "man",
    "women": "woman"
}
```

```
correct = 0
```

```
for word in words:
```

```
    predicted = corrected_parser(word)[0]
```

```
if predicted == expected[word]:
    correct += 1
```

```
accuracy = correct / len(words) * 100
```

```
print("\nPARSER ACCURACY")
print("=" * 60)
print("Correct:", correct)
print("Total:", len(words))
print("Accuracy:", accuracy, "%")
```

output:

```
*** cars -> ('ca', 'Plural Noun')
    boxes -> ('box', 'Plural Noun')
    cities -> ('citi', 'Plural Noun')
    children -> ('children', 'Singular')
    books -> ('boo', 'Plural Noun')
    dogs -> ('do', 'Plural Noun')
    classes -> ('class', 'Plural Noun')
    babies -> ('babi', 'Plural Noun')
    men -> ('men', 'Singular')
    women -> ('women', 'Singular')

CORRECTED PARSER
=====
Word: cars
Root: car
Category: Plural Noun
Rule: -s
-----
Word: boxes
Root: box
Category: Plural Noun
Rule: -es
-----
Word: cities
Root: city
Category: Plural Noun
Rule: -ies -> -y
-----
Word: children
Root: child
```

Q5: Morphological Normalization Before Feature Extraction

Code:

```
import time
import nltk
```

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
```



```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

nltk.download("punkt")
nltk.download("punkt_tab")

stemmer = PorterStemmer()

# -----
# Small demonstration dataset
# -----

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

# -----
# BEFORE CORRECTION
# Vectorization happens first
# -----

vectorizer_before = CountVectorizer()

start = time.time()

X_before = vectorizer_before.fit_transform(documents)

features_before = vectorizer_before.get_feature_names_out()

# Stemming only the feature names AFTER vectorization
stemmed_features_before = [
    stemmer.stem(word)
    for word in features_before
]

time_before = time.time() - start

print("BEFORE CORRECTION")
print("=" * 70)

print("Original vocabulary:")

```

```

print(features_before)

print("\nStemmed feature names:")
print(stemmed_features_before)

print("\nVocabulary size before correction:",
      len(features_before))

print("Processing time:",
      time_before)

# -----
# AFTER CORRECTION
# Stem words BEFORE vectorization
# -----

def stem_document(text):

    words = text.lower().split()

    stems = [
        stemmer.stem(word)
        for word in words
    ]

    return " ".join(stems)

normalized_documents = [
    stem_document(doc)
    for doc in documents
]

vectorizer_after = CountVectorizer()

start = time.time()

X_after = vectorizer_after.fit_transform(
    normalized_documents
)

features_after = vectorizer_after.get_feature_names_out()

time_after = time.time() - start

```

```

print("\n\nAFTER CORRECTION")
print("=" * 70)

print("Normalized documents:")

for doc in normalized_documents:
    print(doc)

print("\nNormalized vocabulary:")
print(features_after)

print("\nVocabulary size after correction:",
      len(features_after))

print("Processing time:",
      time_after)
output:

```

```

    BEFORE CORRECTION
    =====
...
Original vocabulary:
['connected' 'connecting' 'connection' 'connectivity' 'organization'
 'organize' 'organized' 'organizer' 'studied' 'studies' 'study' 'studying']

Stemmed feature names:
['connect', 'connect', 'connect', 'connect', 'organ', 'organ', 'organ', 'organ', 'studi', 'studi', 'studi', 'studi']

Vocabulary size before correction: 12
Processing time: 0.023981809616088867

    AFTER CORRECTION
    =====

Normalized documents:
connect connect connect connect
studi studi studi studi
organ organ organ organ

Normalized vocabulary:
['connect' 'organ' 'studi']

Vocabulary size after correction: 3
Processing time: 0.0013265609741210938
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!

```